

Análisis y Diseño con Patrones

PEC 1: Principios de Diseño y Patrones de Análisis

Pregunta 1 (20%)

Supongamos que estamos desarrollando un software que implementa un sistema similar a una hoja de cálculo. En concreto, queremos que el sistema pueda expresar fórmulas de tres tipos diferentes:

- Un valor literal (1, 2, 3, etc.)
- Una referencia a una celda de la hoja de cálculo (A1, B4, C2, etc.)
- Una función que recibe un número arbitrario de valores (todas las funciones reciben un número arbitrario de valores y, por tanto, no se quiere limitar la aridad de las funciones).

Por ejemplo, estas serían fórmulas válidas:

- =2
- =A1
- =SUMA (2, A1, 3)
- =SUMA (MULTIPLICA (A1,2), MULTIPLICA (B1,3), 4, C2)

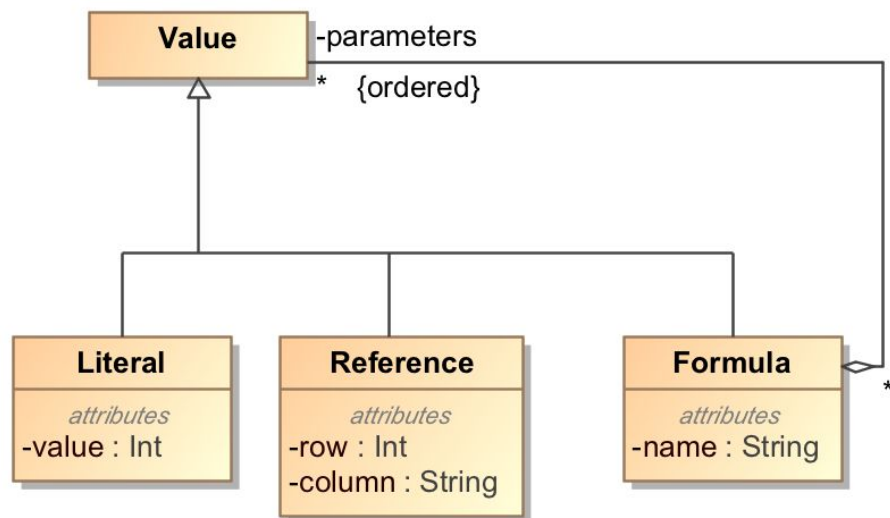
Nos proponemos desarrollar el módulo que implementa este motor de fórmulas. Antes de hacer el diagrama estático de análisis, sin embargo, revisemos los patrones de análisis buscando uno que pueda ayudar a resolver el problema comentado.

Indica qué patrón de análisis habría que tener especialmente en cuenta y justifica brevemente el por qué. Propón un diagrama estático de análisis donde se aplique el patrón de análisis identificado a los requisitos descritos.

Nota: Si es necesario, puedes hacer alguna variación sobre el patrón tal y como está explicado en los materiales para poder satisfacer todos los requisitos enumerados.

Solución

En este caso podemos aplicar el patrón "Objeto compuesto" (*Composite*) ya que nos permite que las fórmulas de tipo función traten todos los argumentos de manera uniforme, ya sean valores literales, referencias a celdas o bien otras funciones.



Hemos usado una agregación UML en vez de una composición porque hemos considerado que un mismo valor (**Value**) puede estar asociado a varias fórmulas. Una composición hubiera sido correcta igualmente ya que no tenemos suficiente información para decantarnos por una opción u otra.

También hemos asumido que todos los literales son enteros. Se podría añadir una clase **LiteralX** (por ejemplo, **LiteralString**) para cada tipo de literal que queramos soportar pero hemos considerado que con esta simplificación era suficiente para mostrar cómo se aplicaría el patrón.

Pregunta 2 (20%)

Imaginemos que estamos desarrollando un software que permite calcular tamaños de elementos de interfaz gráfica a partir de expresiones compatibles con un subconjunto de CSS. En concreto, el subconjunto de CSS que queremos aplicar es el siguiente:

- 2mm (2 milímetros)
- 3px (3 píxeles o $3 * 1/96$ pulgadas)
- 4cm (4 centímetros)
- 1in (96 pixels, 2,54 cm)

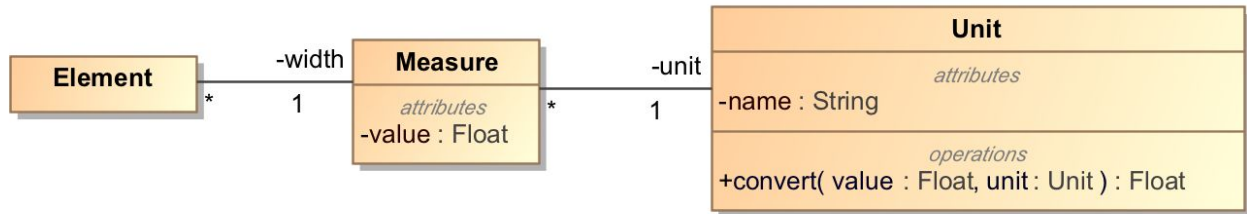
Nos gustaría que, en un futuro, se pudiera añadir el resto de unidades de CSS sin tener que hacer grandes cambios en el sistema. También nos interesa poder "normalizar" los diferentes valores a una unidad de referencia interna del sistema (que podrían ser los píxeles reales, teniendo en cuenta la densidad de píxeles de la pantalla, o cualquier otra unidad que decidiremos cuando lleguemos a implementar esa parte del sistema).

Indica qué patrón de análisis habría que tener especialmente en cuenta y justifica brevemente el por qué. Propón un diagrama estático de análisis donde se aplique el patrón de análisis identificado los requisitos descritos.

Nota: Si es necesario, puedes hacer alguna variación sobre el patrón tal y como está explicado en los materiales para poder satisfacer todos los requisitos enumerados.

Solución

En este caso podemos aplicar el patrón "Cantidad" ya que queremos que la unidad de medida sea explícita y poder utilizar diferentes unidades de medida, así como poder hacer conversiones fácilmente (es decir, nos encontramos ante el problema que resuelve este patrón).



En este caso hemos aplicado una variación respecto a los ejemplos de los materiales donde se tiene en cuenta el hecho de que queremos poder convertir entre unidades. Esto nos obliga a tener una clase Unidad en vez de tratar la unidad como un String tal y como se hacía en los materiales. Esta solución (unidad como String) es también correcta para este ejercicio.

También podríamos haber elaborado un poco más la solución añadiendo información al modelo sobre el ratio de conversión entre unidades. En este caso, sin embargo, hemos considerado que esta información puede ser parte de la implementación de la operación convertir, ya que el número de unidades a soportar lo conocemos en el momento de diseñar la solución (y no variará en tiempo de ejecución).

Pregunta 3 (30%)

Uno de los principios de diseño que debe cumplir un sistema para obtener un nivel alto de calidad es el principio "Abierto / Cerrado" que dice que "Una entidad de software (clase, módulo, función, etc.) debería estar abierta a la extensión pero cerrada a la modificación".

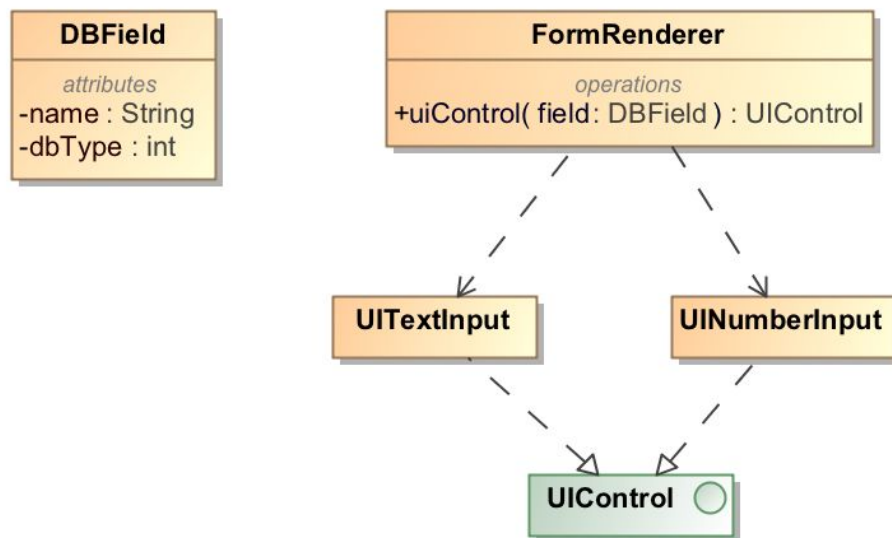
Supongamos ahora que estamos diseñando un sistema que debe permitir editar por pantalla los datos que hay en una tabla de la base de datos. En concreto, hay que utilizar un control de interfaz gráfica diferente en función del tipo de dato. También habrá que hacer unas validaciones diferentes en función del tipo de dato.

Por ejemplo, si tenemos un campo de tipo "integer" debemos validar que el valor introducido por el usuario el control de interfaz `UITextField` esté dentro del rango válido de valores para el tipo "integer" de la base de datos. En cambio, si tenemos un campo de tipo `varchar(10)`, se debe usar un control de tipo `UITextField` y limitar el valor a 10 caracteres de longitud.

Una posible implementación del método que devuelve el control de interfaz gráfica a partir del tipo del campo sería la siguiente:

```
public UIControl uiControl (DBField field){
    switch(field.dbType){
        case Types.VARCHAR:
            return new UITextInput ();
        case Types.INTEGER:
            return new UINumberInput ();
        default:
            throw new IllegalArgumentException ( "Unknown fecha type" + field.dbType);
    }
}
```

En un diagrama de clases, podríamos representarlo como:



- Este ejemplo constituye o puede constituir una violación del principio abierto / cerrado. ¿Por qué?
- ¿Qué consecuencias tendría esta violación?

Solución

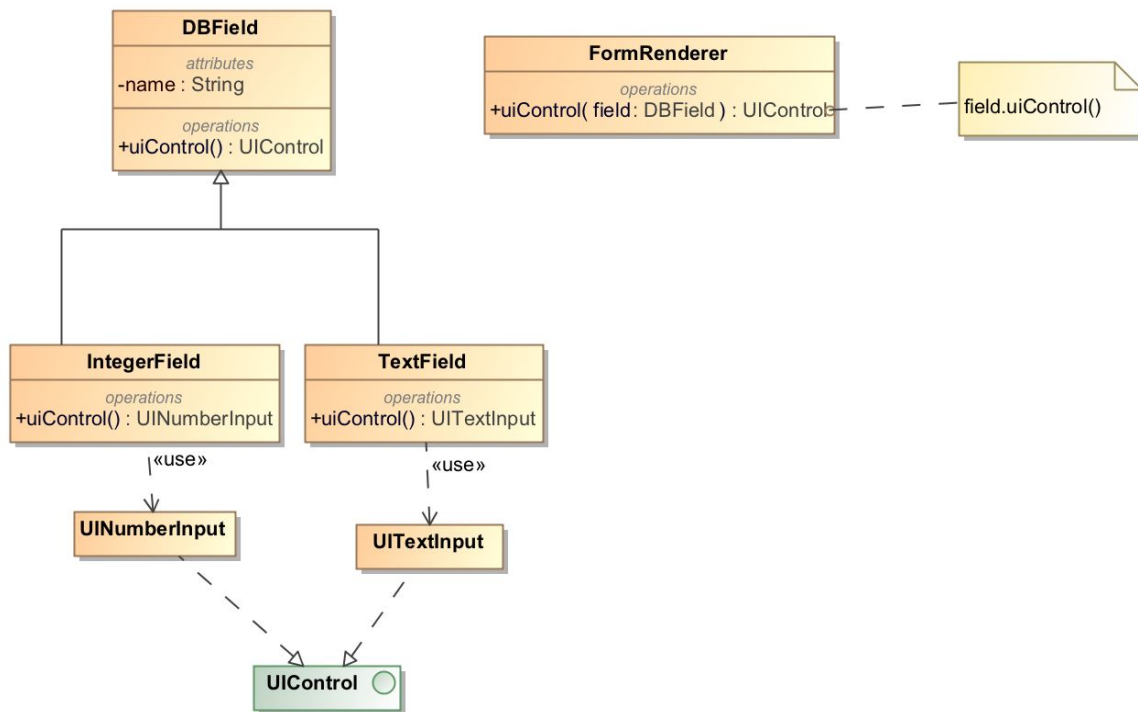
- a) Se trata de una violación del principio abierto / cerrado ya que para añadir un nuevo tipo de control tenemos que modificar el código de la clase *FormRenderer*. Del mismo modo, para introducir nuevas validaciones se deberá modificar la clase *FormRenderer*
- b) La principal consecuencia es que será difícil hacer evolucionar el sistema en este sentido (añadir nuevos tipos de dato) ya que cada vez que queramos añadir un nuevo tipo de dato tendremos que modificar el método *uiControl* así como el método donde se configuren las validaciones (que no se muestra en el enunciado pero sabemos que necesitaremos).

Pregunta 4 (30%)

Propón cómo se podría diseñar el sistema anterior para que se satisfaga el principio abierto / cerrado.

Usa diagramas estáticos de análisis, diagramas de secuencia y / o pseudocódigo de operaciones según creas más relevante para explicar la solución propuesta. Razona, también, cómo la solución supera las consecuencias negativas del apartado *b* de la pregunta anterior.

Solución



Como podemos ver en esta solución, ahora para añadir un nuevo tipo de campo y su correspondiente tipo de control no es necesario modificar ninguna de las clases de este diseño. Para añadir un campo booleano, por ejemplo, habría que añadir un *BooleanField* y su correspondiente *UIBooleanInput*, sin modificar ninguno de las clases (ni métodos) que ya tenemos en este diseño. Asimismo, las validaciones se podrán hacer sencillamente como parte del diseño de la nueva clase *BooleanField* (o de *UIBooleanInput*). Por lo tanto, este diseño es abierto a la extensión pero cerrado a la modificación, ya que podemos extenderlo sin modificar ninguno de los elementos que ya tiene.